

Minimum Recolors to Get K Consecutive Black Blocks

Problem Overview & Walkthrough

Given a 0-indexed string `blocks` of length `n`, where `blocks[i]` is either 'W' (White) or 'B' (Black), and an integer `k` representing the desired number of consecutive black blocks. The objective is to find the **minimum number of operations** (recolorings from White to Black) required to achieve at least one cluster of `k` consecutive black blocks.

Example Walkthrough: `blocks = "WBBWWBB"`, `k = 7`

Visual Input Sequence:



To get `k = 7` consecutive black blocks, we must maintain a window size of exactly 7 and compute the minimum count of white blocks contained within any such window, as these white blocks will need to be repainted to black.

Approach 1: Sliding Window (Fixed Size)

We can systematically maintain a sliding window of fixed size `k`. First, calculate the number of white blocks ('W') in the initial window of size `k`. Then, slide the window across the string one element at a time, updating the count dynamically by adding the incoming character and removing the outgoing character. This avoids recalculating the sum from scratch.

TIME COMPLEXITY

$O(n)$

SPACE COMPLEXITY

$O(1)$

```

class Solution {
public:
    int minimumRecolors(string blocks, int k) {
        int n = blocks.size();
        int ans = 0;

        for(int i = 0; i < k; i++){
            if(blocks[i] == 'W') ans++;
        }
        int minu = ans;
        if(minu == 0) return 0; // kyunki already mil gaya consecutive k black
        painted wall

        for(int j = k; j < n; j++){
            if(blocks[j] == 'W') ans++;
            if(blocks[j-k] == 'W') ans--;
            minu = min(minu, ans);
        }

        return minu;
    }
};

```

Approach 2: Prefix Count

Instead of dynamically updating sums via a sliding window, we precompute the cumulative number of white blocks up to each position using a prefix array. By storing the number of white blocks present within the first i characters, we can extract the white block count of any arbitrary window of size k in $O(1)$ time using subtraction: $\text{prefixWhite}[r + 1] - \text{prefixWhite}[l]$.

TIME COMPLEXITY

$O(n)$

SPACE COMPLEXITY

$O(n)$

```
class Solution {
public:
    int minimumRecolors(string blocks, int k) {
        int n = blocks.size();
        vector<int> prefixWhite(n + 1, 0);

        for(int i = 0; i < n; i++)
        {
            prefixWhite[i + 1] =
                prefixWhite[i] + (blocks[i] == 'W');
        }

        int minRecolors = INT_MAX;

        // -----
        // Check every window
        // -----
        for(int l = 0; l + k - 1 < n; l++)
        {
            int r = l + k - 1;

            int whiteCount =
                prefixWhite[r + 1] - prefixWhite[l];

            minRecolors =
                min(minRecolors, whiteCount);

            if(minRecolors == 0)
                return 0;
        }

        return minRecolors;
    }
};
```